Jot

Complete Specification

A self-contained journal with readable writing inside.

One moment, one day, a whole month, or an ongoing journal.

Writing syntax and parsing rules
55 method and style profiles
27 paper sizes and custom page geometry
Worked examples, migration and acceptance criteria

Version 1.0.0-draft.2
25 October 2026

Inspired by Fountain. Prepared for Shelbee.

Contents

Jot Complete Specification **2**
The writing comes first . . . 2
Reading this specification . . . 3
1. Goals and conformance . . . 3
2. Files, encoding and document structure . . . 4
3. Deterministic parsing and grammar . . . 6
4. Shared content primitives . . . 7
5. Records, measurements and time . . . 8
6. Tasks, collections and relationships . . . 10
7. Method profiles and composition . . . 11
8. Presentation and optional page layouts . . . 11
9. Privacy, authorship and portability . . . 14
10. Migration from Jot v1.0 . . . 15
11. AST and validation contract . . . 16
12. Conformance fixtures and acceptance criteria . . . 17
13. Method catalogue and worked examples . . . 17
14. Implementation sequence and source register . . . 37
15. JavaScript, diagrams, handwriting and the project roadmap . . . 40

Jot Complete Specification

Version 1.0.0-draft.2 | 25 October 2026 | Proposed standard

A self-contained journal document with human-readable writing inside.

The writing comes first

Jot takes its main inspiration from Fountain: a readable plain-text writing format that an application can interpret and typeset. The `.jot` document contains the writing and all its assets together. Its `journal.txt` member holds the human-readable writing itself. It MAY contain a single moment, one day, a whole month, several years, a topic collection, or an ongoing journal. File boundaries MUST NOT imply date boundaries. The filename MUST NOT be treated as authoritative journal content. A date heading organizes writing inside a document; it never requires a separate daily file.

A writer does not need metadata, JSON, a form, a method declaration, a database, or a structured record. The following is complete journal writing, stored as `journal.txt` inside a `.jot` document, combining several styles across two days:

```
# October 2026

### 2026-10-25 | Sunday

.0830 - KITCHEN

The rain made the room feel quieter. I wanted to remember that.

#### A small plan
[ ] Photograph the connector label @home
o Tea with a friend
- Enough for today: start with one small thing.

#### Gratitude
The kettle was already full. Someone remembered my favorite tea.

#### What I noticed
The shop was loud. Headphones helped more than pushing through.

### 2026-10-26 | Monday

#### Morning pages
I don't know what to write yet, so I'll start here...

= There was more room when I left the plan small.
```

The paragraphs are the journal, not descriptions of external journal entries. Ordinary headings such as “Gratitude” or “Morning pages” provide enough structure for a human. Their wording is never a required magic keyword, and software MUST NOT infer health states or task completion from them. Optional structured records later in this specification exist for reliable queries, measurements, links and interchange. They are an extension layer; they MUST NOT become the default authoring requirement or replace prose with a form.

The method examples below show optional structured representations alongside human writing prompts. An editor SHOULD offer a writing-first view and MAY expose structured properties separately. PDF pages, monthly collections and daily views are renderings or selections of the same writing. Splitting a month into daily files, or joining daily files into a month, is an explicit file operation that preserves text order and stable references.

Reading this specification

This document expands Shelbee's original DotJot v1.0 PDF into a language specification, an interoperable data model, and a catalogue of method profiles. It covers every one of the 46 method/style guides present in `shelbeeely/Journaling/docs/journaling/methods` at commit snapshot `f67a54dd20c9` (full hash in source links), plus complementary profiles. The repository is the source of method coverage; the syntax, profile contracts, examples and implementation requirements below are new Jot design proposals.

This is a complete design within its declared scope, not a claim that every journaling tradition has been enumerated or that a parser fully implements every clause. Profiles adapt practices rather than reproducing commercial workbooks, certifying a clinical instrument, or claiming product compatibility. Medical and psychological profiles specify how to record information; they do not prescribe treatment or assert therapeutic effectiveness. The repository's research claims have not been independently audited for this specification.

MUST, MUST NOT, SHOULD, SHOULD NOT and MAY express required, recommended and optional behavior. Normative sections define implementation behavior. Profile explanations and examples are informative unless explicitly stated otherwise. All personal content in examples is fictional.

Contents

1. Goals and conformance
2. Files, encoding and document structure
3. Deterministic parsing and grammar
4. Shared content primitives
5. Records, measurements and time
6. Tasks, collections and relationships
7. Method profiles and composition
8. Presentation and optional page layouts
9. Privacy, authorship and portability
10. Importing the original outline
11. AST and validation contract
12. Conformance fixtures and acceptance criteria
13. Method catalogue and worked examples
14. Implementation sequence and source register

1. Goals and conformance

The writing inside Jot is readable as ordinary text. A short diary paragraph is valid journal content; the portable `.jot` file packages it with any assets. Structure is progressive: a writer can add dates, task marks, measurements, links or templates only when useful. A single entry MAY combine any number of profiles. The format MUST preserve unsorted thoughts, unfinished sentences, empty days, nonstandard labels and unknown extensions without inventing meaning.

The canonical file extension is `.jot`, containing a ZIP-based self-contained document with `manifest.json`, UTF-8 `journal.txt` and all included assets beneath `assets/`. No companion folder is required. This supersedes the earlier draft assumption that the complete `.jot` file was plain text. The writing syntax remains readable and Fountain-inspired, but a generic text editor cannot display the whole packaged file as prose. Readers SHOULD support importing earlier plain-text `.jot`, `.journal`, `.md` and `.txt` source, identified by content rather than extension alone. Plain-text/Markdown export contains writing only and MUST clearly explain that assets are omitted. No registered MIME type is claimed; implementations may use `application/zip` for the package and `text/plain; charset=utf-8` for extracted writing. This is similar to PDF in being one portable file, not PDF-compatible encoding or fixed-layout behavior.

Three capabilities are separately reportable. **Core Reader** parses structural blocks and preserves all source. **Profile Processor** additionally validates one or more named, versioned profiles. **Renderer** additionally produces a declared screen or print layout. A device supporting only Core MUST remain useful for free writing and capture; it MUST preserve unsupported profile blocks. Claiming Jot compatibility MUST list these capabilities, supported versions, limits and extensions. A file validator MUST distinguish syntax errors, profile issues, unresolved links and presentation warnings.

The format is local-first and independent of cloud services, AI, a particular editor or a specific notebook. Handwriting, scans, audio and art can remain primary sources. Text recognition is optional and MUST NOT silently replace the original.

2. Files, encoding and document structure

2.0 Self-contained document container

A Jot v1 package MUST contain exactly one `manifest.json`, one primary `journal.txt`, and every binary asset needed by the saved document. Asset paths are relative to the package root. Typical contents are `assets/photo.png`, `assets/scan.png` and `assets/reflection.jot-ink.json`. These are internal members, not separate files the user must move beside the `.jot`. The file remains v1 while the draft evolves.

`manifest.json` declares `format: jot`, `version: 1.0`, a source descriptor, an ordered asset descriptor array and optional document metadata. Each file descriptor contains `path`, `mediaType`, `byteLength` and a lowercase hexadecimal SHA-256 digest of the uncompressed bytes. The source path is `journal.txt`. An asset descriptor path begins with `assets/`. Descriptors MUST be unique and correspond to actual files. Every non-directory archive member must be listed, except the manifest itself. Ordinary empty directory records beneath `assets/` are accepted. Source and assets MUST pass byte-length and hash checks before an import replaces an open journal. Hashes detect changed/corrupt bytes; they do not authenticate an author or encrypt the contents.

Readers MUST reject path traversal, absolute paths, ambiguous duplicate paths, symlinks and unsupported encryption/compression before extraction. A reader MUST bound file counts and expanded sizes; it must not silently truncate. The prototype supports ordinary single-volume ZIP with STORE or DEFLATE, no ZIP64, a 64 MiB package/expanded limit, 32 MiB per asset, 1 MiB writing and 256 directory entries. These are declared implementation limits, not a promise of unlimited files. Unknown asset types remain bytes with a readable label, and MUST NOT be executed or automatically loaded into active HTML. Remote resources are never fetched just because a link appears in the writing.

The JavaScript interfaces are separated: `parse(text)` consumes the extracted writing; `pack({source, assets})` creates the portable `.jot`; `unpack(bytes)` verifies and reads one package; `readDocument(bytes)` additionally accepts earlier plain-text sources. Asset objects contain `path`, `mediaType` and `data`. The source parser's offsets refer to UTF-8 bytes of `journal.txt`, not offsets into compressed ZIP bytes. The unmodified writing source and exact asset bytes must survive a pack/unpack round trip.

A Jot application opens the single file, renders its writing and visual assets together, and can export writing separately. For recovery or inspection, a standard ZIP utility can extract the readable writing and original assets. The initial implementation does not promise encryption, archival PDF conformance, font embedding or a fixed appearance in every viewer. Page size and layout remain renderable properties of the journal rather than replacing its editable content.

2.1 Text and whitespace

The internal `journal.txt` source MUST be valid UTF-8; a leading UTF-8 BOM is accepted and retained in lossless mode. Readers accept LF and CRLF line endings and a missing final newline. Canonical writers emit LF and one final newline. Unicode content MUST be preserved without

normalization; identifiers use ASCII. Source offsets count UTF-8 bytes from zero, with end exclusive. Lines and columns in diagnostics are one-based; columns count Unicode scalar values, with a tab counting as one column.

Outside literal blocks, tabs expand to the next multiple of four columns for indentation. Syntax markers are case-sensitive unless stated otherwise. Unknown text is prose, not an error. Blank lines separate paragraphs; consecutive ordinary lines in a paragraph form one paragraph with explicit source newlines retained. A renderer MAY visually reflow them. Two trailing spaces request a hard line break. Literal content never reflows. Where earlier Fountain-inspired rules conflict with ordinary Markdown formatting, the explicit Markdown compatibility rules in section 4.2 take priority in this draft.

2.2 Optional metadata

Metadata is recognized only between a first-line `---` and the next standalone `---`. It is deliberately not YAML. Each nonblank line is `Key: value`, split at the first colon. A key matches `[A-Za-z][A-Za-z0-9 -]*`. Values are trimmed strings; there are no escapes, implicit numbers, arrays or executable tags. Blank values are allowed. Keys are compared case-insensitively after trimming. Repeated keys remain ordered; singleton duplicates produce a warning and the last occurrence supplies the effective value. Unknown keys are preserved. An unclosed opening delimiter is ordinary prose with a warning.

```
---
Jot: 1.0
Document-ID: journal-main
Title: Everyday notebook
Author: writer-a
Timezone: America/Los_Angeles
Language: en
Theme: More room to breathe
Default Grid: dot-5mm
Privacy: private
---

### 2026-10-25 | Sunday

A line about today is enough.
```

`Jot` declares the major/minor language version. Missing version means Core 1.0 syntax in a newly created file, but an importer MUST ask for or explicitly select legacy interpretation for known original-outline sources. A reader MUST warn on an unsupported major version and offer read-only/raw preservation. `Document-ID`, when supplied, MUST match `[A-Za-z0-9][A-Za-z0-9._-]{0,127}`. Authors are local identifiers; display names MAY be stored separately. `Timezone` is an IANA identifier; unknown or missing zones remain unresolved, never silently replaced with the device zone. `Privacy` is `private`, `shared` or `public`, default `private`.

2.3 Headings, entries and periods

Headings start at column zero with one to six `#` characters and a space. Levels convey hierarchy, not fixed calendar meanings. A heading beginning with an exact ISO date (`YYYY-MM-DD`), optionally followed by `| label`, opens a dated entry and closes the previous entry regardless of level. Invalid dates remain headings with a diagnostic. Other headings remain within the current entry until the next dated heading. Content before the first dated heading belongs to an undated preamble entry. Duplicate dates form distinct entries in source order; no automatic merge occurs.

Year, month, week, season and collection headings are descriptive. Structured periods use a `period` record with `start` inclusive and `end` exclusive ISO dates. Week calculations use ISO Monday-based weeks when `calendar: iso-week` is declared. A daily view is a projection over entries, not

a requirement to create a blank file every day. Leap days are valid; an anniversary view MUST omit an absent February 29 rather than silently move its entry.

= begins a synopsis line. Multiple synopses are retained in order. === on its own is a page-break block, never an entry boundary. Heading text, dates and dates embedded in prose are never interpreted as scheduling commands.

3. Deterministic parsing and grammar

3.1 Block precedence

A parser MUST apply these rules in order: optional metadata at byte zero; active literal fence; active record block; new literal fence; new record opening; hidden comment; heading; page break; layout directive; forced scene; explicit voice; synopsis; rapid log; marginalia; centered line; paragraph. A block opener starts a new block even without a preceding blank line. Inline parsing happens only after block recognition. Journal extension markers are anchored to column zero except rapid-log continuation lines. Markdown blocks retain their own indentation rules.

```
document       = [metadata], {block} ;
metadata       = "---", NL, {metadata-line | blank}, "---", [NL] ;
heading        = "#"{1..6}, SP, text ;
synopsis       = "=", SP, text ;
pagebreak      = "===" ;
scene          = ".", nonspace, text ;
voice          = "::voice", SP, text ;
record-open    = ":::record", SP, name, [SP, "#", id] ;
record-close   = ":::" ;
directive      = "@", name, ":", [SP], text ;
task           = "[", state, "]", SP, text ;
state          = " " | "x" | ">" | "<" | "-" | "!" ;
log            = ("-" | "o" | "?" | "!"), SP, text ;
name           = lower, {lower | digit | "-"} ;
id             = alnum, {alnum | "." | "_" | "-"} ;
```

This compact grammar describes recognition, not the entire Unicode text grammar. `NL` is an accepted newline; `SP` is one ASCII space; `nonspace` excludes whitespace and newline. Record IDs use the same length and character restriction as document IDs. A fence is a line of at least three backticks, optionally followed by a language label. It closes at a column-zero line containing at least as many backticks and no other nonspace characters. Tilde fences are accepted under the Markdown rules, with the same marker and minimum closing length. Embedded syntax inside fences is always literal.

3.2 Literal escape and recovery

A backslash before the first marker at column zero suppresses block recognition for that line and displays the marker literally. Inline \ escapes the next ASCII punctuation character; other backslashes remain text. A code span enclosed in single backticks suppresses inline interpretation; a backtick that has no partner is ordinary text. To quote longer text containing backticks, use a fenced block with a longer delimiter.

Record blocks cannot nest. An opening record encountered inside a record is invalid record content and remains raw. A missing closing `:::` consumes through EOF and produces an error with the raw bytes intact. An unclosed literal fence behaves similarly. Hidden comments start only with standalone `/*` and close at standalone `*/`; they cannot nest. An unclosed hidden comment consumes through EOF with a warning and MUST show a visible editor diagnostic. All invalid constructs MUST be recoverable as raw content and MUST NOT cause data deletion. Unknown directives and record types are valid opaque blocks.

3.3 Fountain-inspired constructs

`.0830 - WORKSHOP` is a forced scene label. The Core Reader stores its whole label and MUST NOT infer an actual timestamp from `0830`. `::voice ME` starts an explicit voice cue; subsequent paragraphs belong to that cue until a heading, scene, voice cue or record boundary. A standalone parenthetical immediately after a voice cue is an optional delivery/context note. Ordinary ALL-CAPS lines and words such as `LOG:`, `ENTRY:`, `INT.` and `EXT.` remain prose in Core; heuristic recognition exists only in the legacy importer. This prevents diary text such as “I AM TIRED” from becoming an unintended character cue.

4. Shared content primitives

4.1 Rapid capture and lists

`[ ]` means open task; `[x]` completed; `[>]` migrated; `[<]` scheduled; `[-]` cancelled. `[!]` is a v1-compatible open task with priority `high`; it is not a separate lifecycle state. `-` is a Markdown list item that may serve as a rapid note; `o` is an event, `?` a question, and `!` an idea. Markers require one following space; bare `[x]` is prose. Two-space-indented nonblank lines continue the preceding rapid-log item, stripped of exactly two spaces for display. Other indents are preserved as text. Nested actionable tasks SHOULD use records and `parent` links; indentation alone MUST NOT create hidden task relationships.

Inline contexts use `@name` and tags use `#name` at a word boundary, with names containing Unicode letters/digits, `_` or `-`. They end at whitespace or punctuation. Jot does not identify email addresses as contexts because `@` must be at the start or preceded by whitespace. `#` alone never starts a tag. Inside a heading, heading recognition wins; inline tags in its text remain possible. Leading `*` is an ordinary prose list marker; use `[!]` or a task record for machine-readable priority. A renderer MAY make simple prose lists without assigning task semantics.

4.2 Formatting and annotations

Markdown is the base writing syntax. Implementations SHOULD use an established CommonMark/GFM parser and declare its version. Support headings, paragraphs, nested ordered/unordered lists, links, images as attachment references, emphasis, blockquotes, fenced/indented code, tables, task lists and strikethrough. `*italic*` and `_italic_` follow Markdown; `**bold**` is bold; `~~text~~` is strikethrough. The original outline’s underline and single-tilde lyric conventions require explicit legacy import and MUST NOT override Markdown by default. Raw HTML is retained as source but is not executed; renderers MUST disable or sanitize it.

`- [ ] task` and `- [x] task` are Markdown task-list items; bare `[ ] task` and `[x] task` are Jot rapid task markers. Ordinary blockquotes beginning `>` follow Markdown. The old `> centered <` convention is legacy-only; a renderer may offer explicit alignment properties. Standard fenced and indented code suppresses journal commands. Setext heading underlines take priority when `===` immediately follows writing; a standalone `===` after a blank line is a Jot page break. Unknown fenced languages remain literal source.

`[[annotation text]]` is marginalia, not a wiki link. It is single-line and ends at the next unescaped `]]`. Links use `[label](target)` with nonempty label and a target containing no unescaped whitespace or parentheses. Internal targets use `#id` or `other.jot#id`; external targets use explicit `https:` or `http:` schemes. Raw web addresses remain readable text. No script execution is permitted. Nested markup within annotations or link labels remains literal. Overlapping delimiters are consumed left-to-right with precedence: escape, code, link, marginalia, emphasis, tags/contexts. No construct starts inside code.

A Markdown blockquote preserves quoted writing; a `quotation` record additionally stores explicit attribution and source fields. Hidden comments are omitted from rendered journal pages but retained by default in editable exports. They are neither encrypted nor safe places for secrets.

4.3 Generic structured record

```
:::record reflection #evening-20261025
method: gratitude-journaling@1
privacy: private
author: writer-a
gratitude: A friend made time for tea.
gratitude: The rain sounded gentle.
gratitude: I repaired something small.
body: |
  There was room for a difficult day and a good moment.
:::
```

A record is an ordered multimap of fields. Each field is `name: value` at column zero. Field names match the `name` grammar. Field keys are lowercase and case-sensitive. Everything after the first colon is a string with surrounding horizontal whitespace removed. Repeated fields represent repeated values in order. There are no implicit booleans or dates at this layer. Empty string, `0`, `false`, `unknown`, and `not-applicable` are different strings. Absence is different from all of them.

`name: |` introduces a multiline field: every subsequent line starts with two spaces, stripped exactly once; a blank content line is two spaces followed by a newline. It ends at the next non-indented line. Lines with `:::` can therefore be included as `::::`. All markers inside a multiline field are literal. Field values that need leading/trailing spaces or the literal value `|` use this multiline form. A malformed field line is retained in the record's `unparsed` content with a diagnostic. Empty records are Core-valid.

Reserved common fields are `method`, `privacy`, `author`, `created`, `modified`, `date`, `at`, `start`, `end`, `body`, `source`, `relation`, and `label`. A record MAY have several `method` fields. IDs are optional for ephemeral notes and required for durable linking, tasks under migration, attachments and cross-device reconciliation. Duplicate IDs within a document are errors; references to duplicates remain unresolved. Identical IDs in different documents are distinct.

5. Records, measurements and time

5.1 Common record types

The Core Reader recognizes a generic record regardless of its name. A Profile Processor interprets `task`, `event`, `reflection`, `prompt`, `response`, `measurement`, `scale`, `habit`, `observation`, `session`, `period`, `collection`, `quotation`, `attachment`, `layout`, `review`, `consent`, `medication`, `correction` and `relation`. Applications MAY add namespaced types using an `x-organization-` prefix. A type name alone does not enforce a profile.

To make the profile contracts usable, every profile has an ID and integer version. A `method: id@1` field selects it. Every catalogue subsection lists its intended record type, required fields for a **complete** record, optional fields and validation notes. Core always accepts partial capture. Missing required profile fields produce a `profile-incomplete` warning, never rejection of the note. Unless explicitly repeatable, fields in a complete profile are singleton; duplicates warn and their last value supplies the effective value. `body`, `source`, `relation`, `label`, `author`, `privacy`, `created` and `modified` are available to every profile. Common record IDs are separate from profile fields.

5.2 Measurements and scales

```
:::record scale #energy-scale
name: energy
kind: ordinal
minimum: 0
maximum: 5
anchor-low: No usable energy
```

```
anchor-high: Plenty for my usual day
:::

:::record measurement #energy-am
method: symptom-tracking@1
metric: energy
value: 2
scale: #energy-scale
at: 2026-10-25T08:30:00-07:00
status: observed
:::
```

A structured numeric value is an ASCII decimal without grouping, exponent or NaN: `-?[0-9]+(\.[0-9]+)?`. A measurement has `metric`, `value` and optionally `unit`, `scale`, `at`, `status`, `aggregation`, `precision` and `body`. `status` is `observed`, `estimated`, `unknown`, `skipped` or `not-applicable`. For the final three states, omit `value`. Profile contracts MUST allow this explicitly in measurement-based records. Zero is an observation, never a missing value. Repeated observations at different times remain separate records. A unit is an explicit string such as `h`, `min`, `mL`, `mg` or `oz`; no automatic conversion is permitted without a declared conversion and original-value retention. Identically named metrics with different scales MUST NOT be merged as comparable data.

A scale declares `name` and `kind` (`ordinal`, `numeric` or `categorical`). Numeric and ordinal scales declare `minimum` and `maximum`, with optional anchors. Categorical scales repeat `option` in display order. A reference to a missing scale is unresolved rather than defaulted. A missing measurement MUST remain visually distinct from the lowest value. Multiple mood observations on one day require a declared daily aggregation (`latest`, `mean`, `minimum`, `maximum` or `user-selected`) before a year-in-pixels view is generated; mean applies only to numeric scales. If no aggregation is chosen, display multiple markers or no single-day aggregate.

Inline state tags such as `[Energy: High]` are display annotations only. They MUST NOT become numeric measurements without an explicit import mapping. Legacy `Telemetry:` and `Theme-Check:` blocks require conversion into records. A free-form `telemetry` record MAY preserve source text while the author decides its mapping.

5.3 Dates, times and duration

Dates use valid Gregorian `YYYY-MM-DD`. A complete instant uses an RFC 3339 timestamp with seconds and explicit offset or `Z`. Local timestamps lacking an offset MAY be stored as capture text but remain unresolved. A timezone field never resolves a repeated daylight-saving hour without an offset. An offset inconsistent with a declared IANA zone produces a warning; the written instant is preserved. A plain `HH:MM` is display-only unless a profile also supplies date and timezone and ambiguity is resolved explicitly.

Intervals use `start` and `end`; `end` is exclusive and MUST be later than start for a completed timed interval. Overnight intervals use full dates. Ongoing intervals omit `end` with `status: ongoing`. Durations use numeric `duration` and `duration-unit: min` or `s`; an inconsistent duration and interval is flagged, not silently corrected. Future dates do not imply reminders. Jot records schedules; actual notifications belong to an application feature with explicit user configuration.

`date` is the logical journal date; `at` is the observation instant; `created` is when it was entered. These MUST remain distinct. Backfilled material MAY use `recall: retrospective` and `confidence: uncertain` rather than pretend it was recorded contemporaneously. Prompt dates, planned dates, actual dates and import dates have separate fields.

6. Tasks, collections and relationships

6.1 Task lifecycle

Rapid logs provide low-friction capture. Records provide durable identity and detailed behavior:

```
:::record task #repair-lamp
method: getting-things-done-paper@1
text: Replace the lamp cable
status: open
context: home
next-action: Photograph the connector label
project: #lamp-project
estimate: 15
estimate-unit: min
:::
```

Task status is `open`, `in-progress`, `waiting`, `scheduled`, `completed`, `cancelled` or `migrated`. `priority` is `low`, `normal` or `high`. `due` is a deadline; `scheduled-for` is a chosen date/time to work; they are not interchangeable. `completed-at` records completion. A scheduled task MUST have `scheduled-for` or remain profile-incomplete. A migrated task MUST have `migrated-to`, targeting the destination task or collection; unresolved targets are visible warnings. Marking a task migrated never marks it completed.

For an actual forward copy, create a new ID, retain `migrated-from` on the new task, and set `migrated-to` on the old. For a single task shown in a later daily view, retain the same ID and create a relation to that day instead of duplicating it. These are distinct migration modes. Automatic carry-forward MUST NOT silently choose one. Completed or cancelled tasks can be reopened only by an explicit edit or correction. A changed task status does not rewrite previous exports.

6.2 Collections and linked knowledge

A `collection` record declares `title` and `kind`, optionally repeating `member` links. `kind` is a user string such as `index`, `future-log`, `project`, `commonplace`, `inbox`, `waiting`, `someday`, `reference` or `insert`. Membership never moves or deletes the original. A generated index links headings and IDs; page numbers are output-specific locators and MUST NOT replace stable references. Cross-file references are relative to the current document, with a fragment naming a record ID. For a multi-document package, paths MUST stay within the package root.

Relations use `relation: kind target`, split on the first space. Standard kinds are `about`, `parent`, `continues`, `responds-to`, `derived-from`, `summarizes`, `revises`, `supports` and `contradicts`. Unknown kinds are preserved. Cyclic parent/migration chains warn; ordinary knowledge links MAY be cyclic. Unresolved references remain visible and MUST NOT prevent writing.

6.3 Reviews, summaries and recurring activity

A `review` record declares `period-start`, `period-end` (exclusive), `scope`, and repeated `source` references. Scopes include `day`, `week`, `month`, `quarter`, `season`, `year` and user labels. Highlights selected for a weekly review SHOULD link to their source entries. User conclusions and generated summaries MUST be distinguishable via `origin: human`, `origin: imported` or `origin: generated`. Generated text MUST name its source selection and remain editable. Absence from a summary does not delete a source entry.

Core has no executable recurrence language. A profile MAY store `cadence: daily`, `weekly`, `monthly` or a human description, but MUST NOT infer missed tasks, fabricate observations, or schedule notifications. A habit observation is `done`, `partial`, `not-done`, `skipped`, `unknown` or `not-applicable`; only the first three describe participation. Missing days do not become `not-done`. Streak calculations, if offered, MUST state how partial/skipped days and denominator selection work and MUST be optional.

7. Method profiles and composition

Profiles are views and validation contracts over shared blocks. They MUST NOT require a new delimiter for each journaling method. Method identity is local to a record; a daily note can contain a sensory check, a work plan, a photo and free writing together. A file-level theme is a default label; it does not turn every paragraph into a theme score.

A profile definition consists of `id`, `version`, `label`, `record-type`, `required`, `optional`, `repeatable`, `constraints`, `suggested-view`, `source`, and `adaptation-notes`. Profiles in this document are version 1. A published revision that changes the meaning or required fields MUST increment the profile version. New optional fields MAY be added within a version if old readers preserve them. Custom methods SHOULD use `x-owner-name` IDs. Application templates are separate, versioned resources; a filled journal MUST retain the questions actually shown, not only a template ID that might later change.

Composition validates every selected profile independently. A combined record is complete only if every selected profile is complete and their field constraints are compatible. If profiles require incompatible record types or meanings, use separate linked records. For instance, put an emotion measurement beside a CBT reflection rather than overload `value` with both a rating and a narrative. An application MUST permit plain text capture even when a form is incomplete.

Prompt records declare `question`, optional `prompt-id`, `prompt-version`, `source`, `theme`, `offered-on`, and `content-note`. Response records use `prompt` as a link plus `answer` or `status: skipped`. A skipped response MUST NOT infer refusal, failure or a negative mood. Sensitive prompts and on-this-day resurfacing are opt-in presentation behavior. A prompt rotation MUST retain the presented wording and selection date, so later template changes do not rewrite history.

8. Presentation and optional page layouts

8.1 Content and layout separation

`@layout:`, `@spread:` and `@grid:` directives are presentation hints, scoped from their position to the next directive of the same name or EOF. Values are strings. They do not alter data, privacy or entry membership. Recognized values are `@layout: dot-grid-5mm`, `@layout: lined`, `@layout: blank`; `@spread: single-page` or `two-page`; and `@grid: normal`, `hidden` or `blackout`. Blackout is a dark visual background, never redaction. A renderer that cannot honor it MUST warn. Unknown values survive round trips.

More precise layouts use a `layout` record with `target`, `paper`, `orientation`, `margin-mm`, `grid-mm`, `gutter-mm`, `font-pt`, `palette` and repeated `panel` fields. Supported initial targets are `screen`, `print` and `eink`. Named and custom paper dimensions follow section 8.1.1. All lengths are positive decimal millimeters, except margins and gutter MAY be zero. Unsupported values produce a warning and an explicit fallback preview.

Visual journaling stores a text description, media link, optional crop and position, and physical-item locator. Layout is not required to express emotional meaning. Functional icons MUST have text labels or alternatives. Color MUST NOT be the only carrier of task state, mood, urgency or identity. Font, line spacing, grid pitch and density MUST be user-adjustable; a method does not mandate tiny writing.

8.1.1 Paper sizes and physical units

Paper size belongs to the rendering configuration. A `.jot` document containing a whole month MAY be rendered on A4, A5, Letter or a custom notebook size without changing its writing or dates. Physical pages MUST NOT determine entry boundaries. Choosing smaller paper reflows and paginates the selected writing; it MUST NOT automatically reduce handwriting space, grid spacing or font size to preserve the old page count.

The companion `paper-sizes.json` defines exact portrait dimensions in millimeters for ISO A0-A10 and ISO B0-B10, plus Letter, Legal, Tabloid, Half-Letter and Executive. ISO B and JIS B are different families; unqualified `B5` is not accepted as a canonical preset. Use `ISO-B5`, or define custom dimensions for another standard. Preset names are case-sensitive. `Ledger` is an accepted input alias for `Tabloid` with landscape orientation; writers emit the canonical size and explicit orientation. Unknown names remain preserved, but a renderer MUST request dimensions or present a clearly labeled fallback rather than silently substitute Letter.

Canonical preset	Portrait width x height (mm)	Typical use
A3	297 x 420	Large planning sheets
A4	210 x 297	Full-size journal pages
A5	148 x 210	Compact notebooks
A6	105 x 148	Small notebooks
A7	74 x 105	Small reference pages
ISO-B5	176 x 250	Intermediate notebook pages
ISO-B6	125 x 176	Compact notebook pages
Letter	215.9 x 279.4	US full-size pages
Legal	215.9 x 355.6	Longer US pages
Tabloid	279.4 x 431.8	Large US sheets
Half-Letter	139.7 x 215.9	US half-size pages
Executive	184.15 x 266.7	Intermediate US pages

For custom paper use `paper: custom` together with positive `width-mm` and `height-mm`. These dimensions describe the portrait base: width MUST be less than or equal to height. A landscape renderer swaps the two axes. A square is valid and unchanged by orientation. Named presets MUST NOT also declare width/height; conflicting dimensions produce a configuration error with no silent override. `orientation` is `portrait` or `landscape`, default portrait. A printer or renderer MAY support a subset of sizes, but MUST publish its limits and validate the selected size before export.

8.1.2 Simple authoring hints and optional detailed records

For common choices, readable hints are enough:

```
@paper: A5
@orientation: portrait
@margin: 12mm
@dot-spacing: 5mm
@dot-diameter: 0.5mm
@dot-color: #bbbbbb

### 2026-10-25

The writing stays the same when I choose another paper size.
```

`@paper` and `@orientation` use the preset and orientation vocabularies above. `@margin`, `@dot-spacing` and `@dot-diameter` accept a nonnegative decimal followed immediately by `mm` or `in`; dot spacing and diameter MUST be greater than zero. One inch is exactly 25.4 mm. `@dot-color` accepts a six-digit hexadecimal RGB color `#RRGGBB`. A renderer converts lengths to millimeters without discarding the original source. Dot diameter MUST be smaller than dot spacing. These directives apply from their position until another directive of the same name or EOF. A geometry change mid-page starts a new output page; it does not split a journal entry.

Detailed layouts use optional records. This custom-size example has the same 5 mm grid as the A5 example:

```
:::record layout #narrow-notebook
target: print
paper: custom
width-mm: 110
height-mm: 210
orientation: portrait
margin-mm: 10
grid-mm: 5
dot-diameter-mm: 0.5
dot-color: #bbbbbb
:::
```

An active print-layout record applies its explicit fields to subsequent writing until another print-layout record. For each property, renderer defaults have the lowest priority; author hints override defaults; fields in the active print-layout record override hints. An explicit export-dialog override has the highest priority and MUST be shown in the preview. Missing fields inherit lower-priority settings. No override rewrites source text without an explicit save operation. A newly selected named paper size clears inherited custom dimensions; a custom size requires both dimensions in the active configuration.

8.1.3 Margins, binding and grid anchoring

`margin-mm` sets all four page-edge insets. Optional `margin-top-mm`, `margin-right-mm`, `margin-bottom-mm` and `margin-left-mm` override individual edges. Insets are measured from the physical page, independent of a printer's imageable area. `binding` is `none`, `left`, `right` or `top`. `gutter-mm` adds space on that binding edge. `mirror-margins: true` swaps left/right margins and a left/right gutter on even numbered pages; odd pages are recto. The page size itself is unchanged. A two-page spread is a preview arrangement of two physical pages, not a doubled sheet size.

Usable width and height are the oriented sheet dimensions minus the applicable margins and gutter. Both MUST remain positive. The renderer MUST check that the chosen font, grid and selected content fit that region. A large margin on A7 can legitimately leave too little room; the renderer reports the problem rather than clipping the text or silently shrinking it.

`grid-area` is `writing-area` (default) or `full-page`. `grid-anchor` is `center` (default) or `top-left`, relative to the chosen area. Dot spacing measures center-to-center distance, independent of paper size. For `top-left`, the first dot center is one half-spacing from each area's starting edge; later centers advance by exactly one spacing. For `center`, place a centered lattice of complete dots with the same half-spacing minimum inset at the edges; any leftover space is divided equally. If the area is smaller than one spacing, it contains no complete grid cell and the renderer warns. Cropped edge dots MUST NOT be presented as full dots.

Dot diameter describes the full dot width, not the radius. A radial-gradient implementation therefore uses half that diameter for its radius. CSS pixel dimensions MUST NOT be used as the physical paper/grid contract: 20 CSS pixels is about 5.29 mm at the CSS reference conversion, not 5 mm. A 5 mm grid stays 5 mm on A4, A5, Letter and custom pages unless the user explicitly changes its pitch.

8.1.4 HTML, PDF and print validation

An HTML renderer SHOULD generate literal physical dimensions for the selected size, for example `@page { size: 148mm 210mm; margin: 0; }` plus a 148 x 210 mm sheet container. Margins then belong inside the sheet container; do not apply the same margins again through `@page`. The renderer may instead use nonzero page margins if its page-box model accounts for them exactly once. Browser viewport dimensions such as `100vw` and `100vh` are not paper sizes. Generate orientation-resolved literal values rather than depending on CSS custom-property substitution in `@page` across print engines.

PDF page boxes MUST match the requested sheet dimensions using 72 points per inch. Print at actual size/100% to preserve physical grid pitch; fit-to-page scaling changes it. Browser headers/footers, background printing and printer nonprintable edges remain separate settings. A renderer should offer a vector-dot PDF route when a browser cannot reliably print backgrounds. A full-page grid on a non-borderless printer may be cropped even when the source page is correct; show this limit in the preview when the printer capabilities are known.

Renderer acceptance cases: A4 portrait and landscape; A5 and A6; Letter and Half-Letter; ISO-B5 distinct from a custom 182 x 257 mm page; a 110 x 210 mm custom page; a square page; mirrored margins; a gutter; oversized margins; unsupported presets; a mid-document paper change; and invariant 5 mm dot pitch across all sizes. A physical calibration proof SHOULD include a labeled 100 mm line and a grid sample. Previewing successfully is not proof that a printer used the correct paper or scaling setting.

8.2 Pagination and overflow

Renderers MUST preserve all selected writing. Overflow modes are `continue`, `additional-page` or `summary-with-source`. Clipping or silently shrinking text below the chosen minimum is forbidden. A renderer MUST show omitted content counts and continuation locators when a summary is explicitly chosen. Print output MUST declare paper size, scale, margins and page numbering. Privacy selection must remove excluded content from the exported document, not merely conceal it in a hidden layer.

8.3 Handwriting and scans

An attachment declares `path`, `media-type`, `alt`, optional `sha256`, `captured-at`, `page-label`, `region`, `physical-location` and `ocr-status`. `region` is a normalized `x,y,width,height` string with each decimal in [0,1] and the rectangle within the page. A transcription is a separate record linked with `derived-from`, with `origin: imported`, `review-status: unreviewed` or `verified`, and an optional confidence note. Handwriting recognition MUST NOT directly complete tasks, confirm doses or overwrite original values without review.

The `.jot` itself is the portable bundle described in section 2.0. Handwriting images and editable stroke data, scans, photos and other attachments live inside it under `assets/`, referenced by readable paths in `journal.txt`. Missing referenced assets must be reported; supplied but unused assets remain included until the user explicitly removes them. A plain-text export is not a self-contained backup. Jot applications must save/restore the included assets along with writing and clearly distinguish an attachment download from downloading the whole document.

8.4 E-ink and physical styles

E-ink views SHOULD minimize full-screen changes, use static navigation and outlines, and provide explicit refresh behavior. Exact display resolution, touch/stylus capability and contrast are device contracts rather than language rules. A monochrome preview is required before declaring a layout suitable for grayscale e-ink. Long notes need overflow navigation, not clipped boxes.

Dutch doors require an explicit persistent panel and per-leaf regions, plus collision and cut-guide checks. A digital pinned panel MAY approximate the interaction but MUST label itself a digital view. Traveler-style inserts map to collections and bundle members; they do not require leather, a fixed size or a brand. Sticker/deco and junk-journal assets preserve layering order and alternative descriptions. A dot grid is one selectable ruling, not a requirement of Jot.

9. Privacy, authorship and portability

Privacy is an intent label, not encryption or an access-control mechanism. A record's effective privacy is the more restrictive of the document and record (`private` > `shared` > `public`). A public record inside a private document therefore stays private for export. To publish an intentional

selection, create a separate sanitized export with a declared target audience; never lower the source document's label silently. Local viewing by its owner is not publication.

A share/export command MUST let the user select content and include/exclude attachments, hidden comments, annotations, metadata and author identifiers. Default sharing excludes private content and hidden comments and strips precise locations. Excluded content MUST be removed from text, PDF layers, metadata, thumbnails, linked assets and manifests, not merely hidden with a black rectangle. The writing and assets inside a `.jot` document, including hidden comments, are accessible to anyone holding the unencrypted file. Encryption, password storage and secure deletion are application responsibilities, and applications MUST state their actual guarantees.

Collaborative writing records authors per block/record and uses separate `response` or `relation` records for replies. A person's words MUST NOT be silently reassigned to another author. Private sections need separate access-controlled storage; labels inside one shared document cannot conceal them. Conflicting edits MUST retain both versions, with a visible resolution step. Device modification times alone MUST NOT decide which journal text survives.

Export to Markdown, HTML, PDF, text, CSV or JSON MUST declare which constructs lose meaning. CSV exports require an explicit row model and selection, preserving units, scale IDs, missing statuses, source IDs and timestamps. HTML and PDF renderers MUST escape untrusted text and reject active content. Apps MUST NOT evaluate embedded code, expand shell expressions or execute template instructions found in a journal. Search and AI features MUST respect the selected source scope; generated inference is never a measured observation.

10. Migration from Jot v1.0

The original PDF provided a syntax outline and an illustrative JSON object, not a full grammar or a published schema. Version 2 therefore has an explicit legacy import step, followed by a change report and preserved original.

Original outline	Expanded v1 draft
Initial unbounded key/value metadata	Wrap only the recognized opening metadata in `---` delimiters; retain uncertain lines as prose.
Fixed year/month/day heading levels	Retain levels; recognize dated entry headings at any level.
`[!]` urgent task	Preserve spelling; normalize to open plus high priority.
Bare * priority signifier	Convert only with explicit adjacency mapping; otherwise preserve text.
ALL-CAPS voices	Offer heuristic candidates; convert accepted ones to `::voice NAME`.
`LOG:`, `ENTRY:`, `INT.`, `EXT.` headings	Offer explicit scene conversion; no automatic Core recognition.
`.0830 - LOCATION`	Preserve as scene text; date/zone needed for a structured instant.
`Theme-Check:` and `Telemetry:`	Convert to records; retain original strings and scale descriptions.
`[Energy: High]`	Preserve display tag; numeric mapping is opt-in.
`[[ ... ]]`	Keep annotation meaning; do not reinterpret as wiki links.
Inline `/* ... */`	Convert to standalone hidden-comment block or literal text with report.
===, emphasis, layout directives	Preserve with the v1 recognition rules; report changed edge cases.

Original outline	Expanded v1 draft
Example remote `$schema` URL	Replace with bundled schema reference; do not assert a hosted registry exists.

A migration MUST NOT invent dates, task targets, health units, timestamps, author identities or ratings. An original-outline scheduled/migrated marker without a target remains incomplete. Reversible import includes original file hash, selected source dialect, importer version and a list of transformations. No source file is deleted after conversion.

11. AST and validation contract

11.1 Lossless and semantic layers

The normative interchange is a source-preserving JSON envelope. The companion `jot-ast.schema.json` validates that envelope and structural node shapes. It does not validate clinical instruments, dates, cross-references or every profile. Those are semantic validation rules in this document. A structural schema success MUST NOT be reported as full journal conformance.

The root has `format: jot`, `version: 1.0`, original `source`, ordered `metadata`, ordered flat `nodes`, and ordered `diagnostics`. Each node has `type`, byte `start`, byte `end` and `raw`. Record nodes also have `recordType`, optional `id`, ordered `fields`, and `unparsed`. Field objects have `name` and decoded string `value`. Heading nodes have `level` and `text`; paragraph and other text blocks have `text`. A derived entry index MAY reference node ranges, but MUST NOT duplicate or replace source ownership. Semantic caches and views are rebuildable.

Every node's `raw` MUST equal the UTF-8 slice `source[start:end]`. Nodes are contiguous, non-overlapping and cover the document body after the metadata region, including blank-line nodes. Metadata includes its own source range in `metadataRange` when present; without metadata, coverage starts at byte zero. The AST retains every byte, including line endings, unknown blocks and malformed content. Normalized semantic edits may change bytes; untouched nodes MUST remain identical in lossless mode. Canonical formatting is a separate explicit operation.

Diagnostics have `severity` (`info`, `warning`, `error`), stable `code`, `message`, `start` and `end`. Minimum codes: `unsupported-version`, `unclosed-fence`, `unclosed-record`, `unclosed-comment`, `invalid-field`, `invalid-date`, `duplicate-id`, `duplicate-singleton`, `unresolved-reference`, `profile-incomplete`, `profile-conflict`, `invalid-measurement`, `ambiguous-local-time`, `unsafe-path`, `missing-asset` and `layout-overflow`.

11.2 Semantic validation checklist

Validate Gregorian dates, timestamp offsets, range order, profile ID/version availability, profile required fields, numeric/scale bounds, categorical membership, repeatability, task lifecycle requirements, IDs and links. Missing profile definitions are warnings and opaque preservation, not syntax errors. Reject unsafe bundle paths before reading assets. Unknown fields remain valid unless they violate a declared profile constraint. Validators SHOULD provide actionable messages that point to the original line and preserve unsaved text.

Resource limits MUST be published. A Core Reader SHOULD support at least 1 MiB of text, 64 KiB lines and 10,000 records; smaller devices MAY stream or page larger documents. Limits MUST cause a visible partial/read-only state rather than truncation or successful-save claims. There are no remote includes or automatic expansion features. Parsing SHOULD run in linear time in source length using bounded delimiter scanning.

12. Conformance fixtures and acceptance criteria

The following cases are normative input/expected-behavior pairs. They form a starter conformance suite, not evidence of an implemented parser. The companion `conformance.json` makes them machine-readable.

Case	Expected behavior
A single prose sentence, no metadata/date	Valid undated paragraph.
`[!] Phone the repair shop`	Open task, high priority.
`[x]` without following space/text	Ordinary prose.
`I AM TIRED` between blank lines	Prose, no voice inference.
`[[Remember the spare key]]`	Annotation, no link.
`[earlier](#day-am)`	Internal link, unresolved if ID absent.
`[ ] literal` inside a code fence	Literal, no task.
Escaped `\[ ] literal`	Prose showing `[ ] literal`.
Two headings with the same date	Two ordered entries.
`### 2026-02-30`	Heading retained; invalid-date diagnostic.
Measurement value `0`, minimum `0`	Observed zero, not missing.
Measurement status unknown, absent value	Unknown, not observed zero.
Closed record with unknown type/field	Valid opaque preservation.
Missing closing record delimiter	Raw retained through EOF; error.
Duplicate record IDs	Error; references unresolved.
Migration marker with no target	Capture valid; lifecycle incomplete warning.
Local time in a repeated DST hour, no offset	Unresolved instant; no guessed offset.
Attachment `../private.txt` in a bundle	Unsafe path rejected.
Export excluding private record	No private bytes in output or packaged assets.
Source with CRLF and trailing spaces	Lossless round trip is byte-identical.

Before claiming an implementation complete: exercise those cases plus empty files, BOMs, emoji byte offsets, multiline empty values, malformed fields, unmatched inline delimiters, duplicate metadata, nested opener recovery, overnight sessions, leap days, missing assets, out-of-scale ratings, incompatible profiles, interrupted saves and conflicting edits. A renderer additionally needs grayscale, large type, narrow screen, Letter/A4 overflow and export-redaction checks.

13. Method catalogue and worked examples

The following catalogue maps all repository method/style guides to Jot profiles and adds complementary practices. All sample field names are part of the proposed profile contracts. Fields listed as required describe a complete profile record; partial notes remain valid Core. Each example is one record that can be pasted into a `.jot` document. `source` links in the final register locate the repository guide; they are not a claim that Jot exactly implements a branded form.

13.1 ADHD journaling and planning

Profile: `adhd-journaling@1` | Record type: `reflection`

Externalize the next action, time estimate and what actually happened. Allow a tiny restart after any gap, without requiring backlog cleanup.

Writing invitation: What is enough for today? What is the smallest visible first step?

Complete structured record: `next-step`. Optional fields: `estimate`, `actual`, `done`, `distraction`, `restart`, `support`.

A done list and an interruption parking lot are optional. No automatic failure score, compulsory streak, or assumption that an unused planner means low motivation.

```
:::record reflection #example-adhd-journaling
method: adhd-journaling@1
next-step: Put the notebook beside the kettle
estimate: 2 min
done: Opened the notebook
:::
```

13.2 Alastair lists and rolling columns

Profile: `alastair-method@1` | Record type: `task`

Keep one task list and associate items with month, day, context or project-status columns. The columns are a view over items, so an item need not be rewritten for every column.

Writing invitation: Which column helps me find this again: month, day, context, or status?

Complete structured record: `text`, `status`, `column`. Optional fields: `column-kind`, `scheduled-for`, `migrated-to`.

column is repeatable for contexts/days. A project-status view uses a single current status. A month column is not an exact appointment date; retain the exact date separately when known.

```
:::record task #example-alastair-method
method: alastair-method@1
text: Book a repair appointment
status: open
column: November
column-kind: month
:::
```

13.3 Art, junk and sketchbook journals

Profile: `art-and-junk-journaling@1` | Record type: `reflection`

Record a day through drawings, collage, found paper and a few words. The journal can describe a physical page while keeping scans and transcriptions as optional companions.

Writing invitation: What image, scrap, texture or mark belongs to this day?

Complete structured record: `body`. Optional fields: `asset`, `physical-location`, `material`, `layer`, `alt`.

asset, material and layer are repeatable. Physical objects need not be digitized. Preserve layer order and text alternatives; do not claim that a text record reproduces the texture of an original page.

```
:::record reflection #example-art-and-junk-journaling
method: art-and-junk-journaling@1
body: A bus ticket beside a sketch of wet boots.
physical-location: Blue notebook, page 18
:::
```

13.4 Sensory and autistic experience logs

Profile: `autistic-sensory-journaling@1` | Record type: `observation`

Notice sensory context, capacity, communication preferences and supports. Entries can be short words or icons with text labels, without turning regulation into compliance.

Writing invitation: What was the environment like? What helped? What do I need next?

Complete structured record: `context`, `support`. Optional fields: `sensory-load`, `communication`, `masking`, `recovery`, `body`.

support is repeatable. Ratings need an explicit personal scale. A log describes experience; it must not infer a diagnosis, demand reduced stimming or automatically recommend increasing sensory tolerance.

```
:::record observation #example-autistic-sensory-journaling
method: autistic-sensory-journaling@1
context: Busy grocery aisle
support: Headphones and a quieter checkout
communication: Typing preferred
:::
```

13.5 Bullet Journal and rapid logging

Profile: `bullet-journal@1` | Record type: `collection`

Combine tasks, events, notes, a future log and topic collections. Reviews deliberately decide what to carry forward, schedule or release.

Writing invitation: What happened, what needs attention, and what is still worth carrying?

Complete structured record: `title`, `kind`. Optional fields: `member`, `period-start`, `period-end`.

Use ordinary rapid-log lines for everyday writing. member is repeatable when building an index. Migration follows section 6; an unfinished item is not automatically copied or counted as a failure.

```
:::record collection #example-bullet-journal
method: bullet-journal@1
title: October daily log
kind: daily-log
:::
```

13.6 CBT-style thought records

Profile: `cbt-thought-record@1` | Record type: `reflection`

Separate a situation, an automatic thought, feelings, observations and an alternative interpretation. Preserve the original wording instead of silently reframing it.

Writing invitation: What happened? What did I think? What evidence can I actually observe?

Complete structured record: `situation`, `thought`, `emotion`, `evidence-for`, `evidence-against`, `alternative`. Optional fields: `before`, `after`, `scale`, `action`, `outcome`.

evidence-for and evidence-against are repeatable. Before/after ratings require the same declared scale. This is a recording adaptation, not a validated replacement for a particular clinical worksheet; software must not decide that a reported harm is a distorted thought.

```
:::record reflection #example-cbt-thought-record
method: cbt-thought-record@1
situation: A message had no reply this afternoon.
thought: Perhaps I annoyed them.
emotion: Worry
evidence-for: I have not received a reply.
evidence-against: They said this week was busy.
alternative: I do not know yet; there are several explanations.
:::
```

13.7 Commonplace books and linked ideas

Profile: `commonplace-book@1` | Record type: `quotation`

Collect passages, observations and ideas under subjects, with clear attribution and commentary. Subject organization can coexist with chronological diary writing in the same file.

Writing invitation: What is worth keeping, where did it come from, and what does it mean to me?

Complete structured record: `text`, `attribution`. Optional fields: `source`, `locator`, `topic`, `commentary`, `relation`.

topic and source are repeatable. Quotations and personal commentary remain distinct. Links can connect ideas without claiming that a commonplace collection is identical to a full Zettelkasten system.

```
:::record quotation #example-commonplace-book
method: commonplace-book@1
text: Start with the thing you can inspect.
attribution: My own workshop note
topic: Making
commentary: A useful reminder before debugging.
:::
```

13.8 DBT-style diary cards

Profile: `dbt-diary-card@1` | Record type: `observation`

Keep brief, dated records of emotions, target behaviors and skill practice. A weekly card is a view assembled from observations, not a reason to flatten all daily data into one score.

Writing invitation: What did I notice, and what skill or support did I try?

Complete structured record: `date`, `target`, `status`. Optional fields: `value`, `scale`, `skill`, `effect`, `body`.

Use locally configured fields and scales from the actual care plan. No automatic clinical risk classification or unsolicited disclosure. Skill-use and urge ratings must not be conflated. This is an optional representation, not certification of a clinical diary card.

```
:::record observation #example-dbt-diary-card
method: dbt-diary-card@1
date: 2026-10-25
target: Skills practice
status: observed
skill: Grounding with a familiar object
body: It helped me return to the conversation.
:::
```

13.9 Daily notes and periodic reviews

Profile: `digital-daily-notes@1` | Record type: `reflection`

Use date headings, links and optional prompts as navigation. A daily note is a section of writing; a month file and separate day files are equally valid storage choices.

Writing invitation: What should future me be able to find from today?

Complete structured record: `date`, `body`. Optional fields: `template-id`, `template-version`, `relation`.

Do not require a file per date or infer dates from filenames. Automatic templates must leave the actual questions in the writing. Periodic summaries link to sources and preserve unsummarized material.

```
:::record reflection #example-digital-daily-notes
method: digital-daily-notes@1
date: 2026-10-25
body: I learned why the connector was loose.
:::
```

13.10 Dream journals

Profile: `dream-journal@1` | Record type: `reflection`

Capture remembered scenes soon after waking, allowing fragments and uncertainty. Optional interpretation belongs in a separate field from the remembered report.

Writing invitation: What fragments, feelings or images do I remember?

Complete structured record: `report`. Optional fields: `woke-at`, `recall`, `emotion`, `dreamsign`, `interpretation`.

emotion and dreamsign are repeatable. A blank report can be a partial record, or status unknown in a generic observation. Interpretation is the writer's reflection, never a factual inference about other people or a diagnosis.

```
:::record reflection #example-dream-journal
method: dream-journal@1
report: A train station with plants growing through the roof.
recall: Fragments
emotion: Curious
:::
```

13.11 Dutch door pages and persistent panels

Profile: `dutch-door-method@1` | Record type: `layout`

Keep a weekly or reference panel visible while daily leaves change underneath. This is a physical/digital layout style that can present any journaling method.

Writing invitation: What information should remain in view while I turn the page?

Complete structured record: `target`, `persistent-panel`, `leaf-region`. Optional fields: `paper`, `cut-guide`, `member`.

The human-readable region descriptions in this profile are design intent, not cut-ready coordinates. A production renderer must resolve geometry, check safe regions and provide a proof before calling the result printable.

```
:::record layout #example-dutch-door-method
method: dutch-door-method@1
target: print
persistent-panel: Weekly appointments at top
leaf-region: Daily notes below
:::
```

13.12 E-ink journal layouts

Profile: `e-ink-templates@1` | Record type: `layout`

Present writing in calm pages with readable contrast, explicit navigation and room for handwriting. Match the actual device rather than assume every e-ink screen has the same input.

Writing invitation: What should be visible at a glance, and where can longer writing continue?

Complete structured record: `target`, `palette`. Optional fields: `width-px`, `height-px`, `input`, `overflow`, `refresh`.

Pixel dimensions are positive integers. Navigation remains available without animation or color. Template rendering is separate from the .jot source, and handwritten originals stay attached when text is recognized.

```
:::record layout #example-e-ink-templates
method: e-ink-templates@1
target: eink
palette: monochrome
overflow: continue
input: stylus
:::
```

13.13 Urgency and importance sorting

Profile: `eisenhower-matrix@1` | Record type: `task`

Sort tasks on two independent dimensions and choose an action deliberately. A deadline and a personal priority are different properties.

Writing invitation: Is it time-sensitive? Does it matter to my values or commitments?

Complete structured record: `text`, `status`, `urgent`, `important`. Optional fields: `due`, `decision`, `delegate`.

urgent and important are explicit true/false strings; absence means unclassified. Both true suggests do; important only suggests schedule; urgent only suggests delegate/batch; neither suggests release/reconsider. Suggestions never delete tasks.

```
:::record task #example-eisenhower-matrix
method: eisenhower-matrix@1
text: Prepare questions for the appointment
status: open
urgent: false
important: true
decision: schedule
:::
```

13.14 Emergent daily planning

Profile: `emergent-task-planner@1` | Record type: `reflection`

Choose a few important tasks, estimate time and record what emerges. Let interruptions and changing capacity remain part of the day's account.

Writing invitation: What are my few important outcomes, and what actually happened around them?

Complete structured record: `focus`. Optional fields: `estimate`, `actual`, `interruption`, `available-time`.

focus and interruption are repeatable. Time bubbles are a display choice; they must label their duration. A custom layout is an adaptation, not a reproduction of the official planner artwork.

```
:::record reflection #example-emergent-task-planner
method: emergent-task-planner@1
focus: Inspect the power cable
estimate: 15 min
actual: 25 min
interruption: Needed a different screwdriver
:::
```

13.15 Expressive writing sessions

Profile: `expressive-writing@1` | Record type: `session`

Provide private, uninterrupted prose with an optional time boundary and a gentle closing note. The writing can remain entirely unstructured.

Writing invitation: What do I want to put into words, and what would help me close this session?

Complete structured record: `body`. Optional fields: `start`, `end`, `duration`, `duration-unit`, `closing`.

A saved session does not require disclosure, analysis, a minimum word count or an escalating prompt. Duration settings are user choices. Claims about a particular therapeutic protocol require its own evidence and qualified implementation.

```
:::record session #example-expressive-writing
method: expressive-writing@1
body: I want to describe what the change has felt like.
duration: 10
duration-unit: min
closing: Tea and a quieter room
:::
```

13.16 Short morning and evening reflection

Profile: `five-minute-journal@1` | Record type: `reflection`

Use a small morning intention and an evening look-back. Generic, original prompts can support a brief routine without copying a commercial page.

Writing invitation: One good thing, one intention, and one thing I noticed by evening.

Complete structured record: `phase`, `body`. Optional fields: `gratitude`, `intention`, `affirmation`, `highlight`, `adjustment`.

phase is morning or evening; missing either is not a failure. gratitude and highlight are repeatable. No forced positivity, quota or requirement to reframe a difficult day as a good one.

```
:::record reflection #example-five-minute-journal
method: five-minute-journal@1
phase: morning
body: I want a slower start.
gratitude: A warm room
intention: Leave time to eat
:::
```

13.17 Multi-year and line-a-day diaries

Profile: `five-year-diary@1` | Record type: `reflection`

Write a small entry on a real date, then compare the same month/day across years. The multi-year page is a view over ordinary dated writing.

Writing invitation: What small detail from today might I be glad to rediscover?

Complete structured record: `date`, `body`. Optional fields: `prompt`, `memory-visibility`.

Resurfacing is opt-in and excludes private material from shared views. Actual dates are retained; February 29 stays February 29. A short display limit must not truncate the stored writing.

```
:::record reflection #example-five-year-diary
method: five-year-diary@1
date: 2026-10-25
body: Rain on the window while I fixed the lamp.
:::
```

13.18 Goals, weekly outcomes and daily focus

Profile: `full-focus-planner@1` | Record type: `reflection`

Connect a longer-term goal to a weekly outcome and a small daily selection. Add startup/shutdown rituals only when they help.

Writing invitation: Which daily action supports this week's outcome?

Complete structured record: `horizon`, `outcome`. Optional fields: `goal`, `next-action`, `ritual`, `review`.

horizon is day, week, quarter or year. outcome and ritual are repeatable. A top-three view is a configurable preference, not a Core limit. This profile borrows a planning pattern and does not reproduce a branded planner.

```
:::record reflection #example-full-focus-planner
method: full-focus-planner@1
horizon: day
outcome: Understand the cable fault
next-action: Check the printed connector label
:::
```

13.19 Gender euphoria and HRT records

Profile: `gender-euphoria-and-hrt-log@1` | Record type: `observation`

Give moments of rightness, identity and joy their own space. Medication and laboratory records can sit nearby without reducing the journal to a treatment timeline.

Writing invitation: When did I feel like myself? What would I like to remember?

Complete structured record: `category`, `body`. Optional fields: `at`, `medication`, `lab`, `source`.

Keep joy observations separate from medication dose events and lab measurements. No automatic dose suggestions, assumptions about anatomy, or requirement to record dysphoria. Names and identity metadata require careful export selection.

```
:::record observation #example-gender-euphoria-and-hrt-log
method: gender-euphoria-and-hrt-log@1
category: euphoria
body: I liked hearing my name in a familiar voice.
:::
```

13.20 Capture, next actions and weekly review

Profile: `getting-things-done-paper@1` | Record type: `task`

Capture an open loop, clarify its next visible action and organize it by context, project or waiting state. Reviews reconnect lists to current commitments.

Writing invitation: What is this, and what is the next physical action?

Complete structured record: `text`, `status`, `next-action`. Optional fields: `project`, `context`, `waiting-for`, `review-on`, `scheduled-for`.

A project may contain several linked tasks; waiting-for is distinct from delegated ownership. The calendar holds explicit commitments rather than guessed deadlines. Reference material remains separate from actions.

```
:::record task #example-getting-things-done-paper
method: getting-things-done-paper@1
text: Repair the lamp
status: open
next-action: Photograph the connector
context: home
:::
```

13.21 Gratitude and small good things

Profile: `gratitude-journaling@1` | Record type: `reflection`

Name a specific thing appreciated, optionally including why it mattered. An entry can hold gratitude and discomfort together.

Writing invitation: What was a small good thing, without needing the whole day to be good?

Complete structured record: `gratitude`. Optional fields: `why`, `person`, `body`.

gratitude is repeatable. No minimum count or mandatory daily cadence. Avoid making a gratitude entry a condition for saving other writing, and let the writer skip the prompt.

```
:::record reflection #example-gratitude-journaling
method: gratitude-journaling@1
gratitude: Someone saved me a seat.
why: I felt welcome.
:::
```

13.22 Habits and partial participation

Profile: `habit-tracking@1` | Record type: `habit`

Record participation in a behavior with room for partial, skipped and unknown states. A chosen cue can be recorded without claiming that tracking proves a habit is automatic.

Writing invitation: What did I try, and what made it easier to begin?

Complete structured record: `name`, `date`, `status`. Optional fields: `cue`, `amount`, `unit`, `body`.

status uses the habit vocabulary in section 6. Amount is optional numeric data with an explicit unit. Do not treat partial participation, illness, missing data and a deliberate rest day as the same state.

```
:::record habit #example-habit-tracking
method: habit-tracking@1
name: Read a little
date: 2026-10-25
status: partial
body: One page before bed
:::
```

13.23 Daily life-book pages

Profile: `hobonichi-techo@1` | Record type: `reflection`

Mix a dated day page, small timeline, free writing, sketches and ephemera. The style gives the day room without imposing a single method.

Writing invitation: What belongs on this day's page: words, a sketch, a receipt, or nothing yet?

Complete structured record: `date`, `body`. Optional fields: `asset`, `layout`, `timeline`.

A brand-inspired daily view must not imply an exact replica of a commercial page. Grid pitch, paper size and decorative elements are user choices; overflow preserves long entries.

```
:::record reflection #example-hobonichi-techo
method: hobonichi-techo@1
date: 2026-10-25
body: The new tea tasted like oranges.
:::
```

13.24 Interstitial and transition notes

Profile: `interstitial-journaling@1` | Record type: `reflection`

Write a few lines when leaving one activity and entering the next. Preserve where to resume as well as what happened.

Writing invitation: Where did I stop? How am I arriving? What is the first next action?

Complete structured record: `stopped`, `next-step`. Optional fields: `at`, `feeling`, `resume`.

A scene heading can supply a human time cue; an exact instant needs at with offset. A transition note does not claim that the previous task is completed.

```
:::record reflection #example-interstitial-journaling
method: interstitial-journaling@1
stopped: Measured the connector spacing
next-step: Compare the photo with the parts list
resume: Notebook page 18
:::
```

13.25 Short ordered next-day lists

Profile: `ivy-lee-method@1` | Record type: `collection`

Choose a small ranked list for the next work period and revisit it deliberately. Carry-forward keeps task identity and distinguishes unfinished work from newly selected work.

Writing invitation: What few things matter most tomorrow, and in what order?

Complete structured record: `title`, `kind`. Optional fields: `for-date`, `member`, `limit`.

member is repeatable and order is meaningful. The traditional six-item limit can be a view setting; a smaller capacity-based list is an explicitly labeled adaptation. Reordering does not change deadlines.

```
:::record collection #example-ivy-lee-method
method: ivy-lee-method@1
title: Tomorrow
kind: ranked-actions
for-date: 2026-10-26
limit: 6
:::
```

13.26 Lifelog plus lasting reference

Profile: `jibun-techo@1` | Record type: `collection`

Separate day/week records from lasting life reference and loose ideas. A 24-hour view can show both planned and actual activity.

Writing invitation: Is this for the current diary, lasting reference, or an idea I want to keep?

Complete structured record: `title`, `kind`. Optional fields: `member`, `horizon`, `layout`.

A collection need not be a separate file. Distinguish plan and actual intervals; keep overnight activity on actual dates. Do not copy brand artwork or imply exact page compatibility.

```
:::record collection #example-jibun-techo
method: jibun-techo@1
title: Things I want to learn
kind: life-reference
horizon: ongoing
:::
```

13.27 Mood observations and year views

Profile: `mood-tracking-year-in-pixels@1` | Record type: `measurement`

Keep a mood label or rating with context, then generate a yearly view only from an explicit daily selection or aggregation rule.

Writing invitation: Which word or rating describes this moment, on my own defined scale?

Complete structured record: `metric`, `status`. Optional fields: `value`, `scale`, `at`, `aggregation`, `body`.

A categorical scale may be inline human documentation or a linked scale record; automated comparison requires a linked scale. value is required when observed or estimated. A year view must label missing days and retain multiple daily observations.

```
:::record measurement #example-mood-tracking-year-in-pixels
method: mood-tracking-year-in-pixels@1
metric: mood
status: observed
value: calm
body: A word label, not a diagnostic score.
:::
```

13.28 Morning Pages and freewriting

Profile: `morning-pages@1` | Record type: `session`

Write continuously in ordinary paragraphs, without judging style or solving everything. Longhand and a three-page practice can be tracked as a personal ritual without becoming a file constraint.

Writing invitation: Start with whatever is present, including not knowing what to write.

Complete structured record: `body`. Optional fields: `date`, `medium`, `page-count`, `closing`.

A typed or shorter session is a labeled adaptation if the writer wants to distinguish it from the original practice. No automatic sentiment scoring, summary or resurfacing of private freewriting.

```
:::record session #example-morning-pages
method: morning-pages@1
body: I woke up with three unrelated ideas and no need to sort them yet.
medium: handwriting
:::
```

13.29 Paper, ruling and page geometry

Profile: `paper-and-grids@1` | Record type: `layout`

Choose blank, lined, square, dot or cue-column ruling to fit the writer and output. Layout records preserve choices without altering the words.

Writing invitation: What spacing lets me write comfortably?

Complete structured record: `target`, `ruling`. Optional fields: `grid-mm`, `line-mm`, `paper`, `margin-mm`, `font-pt`, plus the paper, orientation, dot, binding and per-edge margin settings in section 8.1.

ruling is blank, lined, square, dot or cornell. Positive lengths and usable-area checks are required before rendering. Printer performance and paper behavior need an actual proof; the format cannot guarantee physical ink behavior.

```
:::record layout #example-paper-and-grids
method: paper-and-grids@1
target: print
ruling: dot
grid-mm: 5
paper: Letter
margin-mm: 12
:::
```

13.30 Roadmaps and values-led planning

Profile: `passion-planner@1` | Record type: `reflection`

Connect a future possibility to a concrete next step, with space for personal and work focus and a monthly look-back.

Writing invitation: What would I like more of, and which step would bring it closer?

Complete structured record: `horizon`, `aspiration`, `next-step`. Optional fields: `domain`, `milestone`, `due`, `not-to-do`, `review`.

milestone and not-to-do are repeatable. Aspirations are not obligations, and a calendar date is not inferred from a horizon phrase. This is a general roadmap adaptation, not a reproduced commercial worksheet.

```
:::record reflection #example-passion-planner
method: passion-planner@1
horizon: three months
aspiration: Make more small creative things
next-step: Reserve one quiet evening
:::
```

13.31 Functional stickers and decorative journaling

Profile: `planner-sticker-and-deco-culture@1` | Record type: `attachment`

Let decoration carry memory or aid navigation, while functional stickers can label a section or event. Treat artwork as optional rather than a prerequisite for using the journal.

Writing invitation: Is this decoration, a memory, or a symbol that carries information?

Complete structured record: `path`, `media-type`, `alt`, `role`. Optional fields: `relation`, `layer`, `license`, `creator`.

role is decorative or functional. Functional assets need a text equivalent; decorative assets may have a brief descriptive alternative. An example path is illustrative and not a claim that the asset is bundled.

```
:::record attachment #example-planner-sticker-and-deco-culture
method: planner-sticker-and-deco-culture@1
path: assets/rain-sticker.png
media-type: image/png
alt: A small rain cloud
role: decorative
:::
```

13.32 Focus rounds and interruption logs

Profile: `pomodoro-log@1` | Record type: `session`

Plan a bounded work round, record its actual outcome and park interruptions. Keep the task and the round as separate concepts.

Writing invitation: What will this round focus on, and what interrupted it?

Complete structured record: `focus`, `status`. Optional fields: `planned-duration`, `duration`, `duration-unit`, `start`, `end`, `interruption`.

status is planned, ongoing, completed or interrupted. interruption is repeatable. A completed round does not complete the linked task. Alternate round lengths are labeled adaptations, and the journal does not require a running timer.

```
:::record session #example-pomodoro-log
method: pomodoro-log@1
focus: Read the connector datasheet
status: completed
planned-duration: 25 min
duration: 25
duration-unit: min
:::
```

13.33 Life review and written dialogue

Profile: `progoff-intensive-journal@1` | Record type: `reflection`

Explore a period of life through turning points, dialogue and linked sections. A new observation can lead to another section without forcing a single chronological narrative.

Writing invitation: What period am I in, and what other part of my life does this connect to?

Complete structured record: `section`, `body`. Optional fields: `relation`, `voice`, `period-start`, `period-end`.

This is a general life-review adaptation, not a replacement for the official Intensive Journal workbook or workshop sequence. The writer controls pace and interpretation. Dialogue voices can use explicit voice cues in ordinary writing.

```
:::record reflection #example-progoff-intensive-journal
method: progoff-intensive-journal@1
section: Turning points
body: The move changed which routines felt like mine.
:::
```

13.34 Prompt-based writing

Profile: `prompt-journaling@1` | Record type: `response`

Start from a chosen question and keep the question with the answer. A prompt may be answered, adapted, postponed or skipped.

Writing invitation: What would help me notice something specific today?

Complete structured record: `question`, `status`. Optional fields: `answer`, `prompt`, `prompt-version`, `source`.

status is answered, partial, skipped or postponed. answer is required only when answered. A linked prompt is optional when question is stored locally; changing a prompt template must not change an existing response.

```
:::record response #example-prompt-journaling
method: prompt-journaling@1
question: What made the room feel welcoming?
status: answered
answer: A lamp instead of the overhead light.
:::
```

13.35 Paper scans and transcription

Profile: `rocketbook-and-scan-systems@1` | Record type: `attachment`

Identify a physical page and keep its scan, optional transcription and routing choices linked. The paper original remains a legitimate source.

Writing invitation: Which page is this, where is the original, and has the text been checked?

Complete structured record: `path`, `media-type`, `alt`. Optional fields: `page-label`, `physical-location`, `ocr-status`, `captured-at`, `sha256`.

ocr-status is none, unreviewed or verified. Brand routing symbols and encoded dot patterns are not assumed interoperable. A printed page ID should encode a local opaque identifier, not private journal content.

```
:::record attachment #example-rocketbook-and-scan-systems
method: rocketbook-and-scan-systems@1
path: assets/page-018.jpg
media-type: image/jpeg
alt: Handwritten notes and a cable sketch
page-label: 18
ocr-status: unreviewed
:::
```

13.36 Personal support and safety-plan records

Profile: `safety-planning@1` | Record type: `collection`

Represent an existing, author-chosen support plan as a stable, accessible section. Preserve its intended order, review date and contact details without automatic crisis inference.

Writing invitation: What support plan have I chosen, and where can I find the current copy?

Complete structured record: `title`, `kind`. Optional fields: `warning-sign`, `coping`, `contact`, `environment`, `reviewed-on`, `member`.

warning-sign, coping, contact, environment and member are repeatable. This data profile is not a complete clinical plan template or emergency service. Actual contacts and steps come from the author's existing plan; no alerts or sharing occur just because the record exists.

```
:::record collection #example-safety-planning
method: safety-planning@1
title: My support plan
kind: support-plan
reviewed-on: 2026-10-25
```

```
coping: Sit in the familiar quiet room
:::
```

13.37 Curiosity and growth reflections

Profile: `shadow-growth-journal@1` | Record type: `reflection`

Notice a reaction, possible meaning and a compassionate next step. Interpretations remain tentative and can be revised without overwriting the original feeling.

Writing invitation: What did I notice in my reaction, and what else could be true?

Complete structured record: `reaction`, `body`. Optional fields: `hypothesis`, `strength`, `boundary`, `closing`.

Do not infer hidden motives, recovered memories or personality diagnoses. A prompt can be skipped; the software must not insist that mistreatment is a projection or the writer's fault.

```
:::record reflection #example-shadow-growth-journal
method: shadow-growth-journal@1
reaction: Envy when a friend finished a project
body: I may want more protected making time.
strength: I care about creating things.
:::
```

13.38 Spoons and personal energy accounting

Profile: `spoon-theory-energy-accounting@1` | Record type: `observation`

Describe felt capacity, estimated costs and helpful recovery in a personally meaningful unit. The account is a metaphor and a planning aid, not a physiological measurement.

Writing invitation: What capacity do I feel I have, and where would I like to spend it?

Complete structured record: `unit`, `status`. Optional fields: `available`, `cost`, `recovery`, `balance`, `at`, `body`.

Free-text costs are valid reflection. Arithmetic views require numeric cost/recovery fields in separate measurement records with explicit scope. Never compare spoon counts between people or automatically increase activity targets.

```
:::record observation #example-spoon-theory-energy-accounting
method: spoon-theory-energy-accounting@1
unit: personal-spoons
status: estimated
available: 4
cost: Shopping felt like 2
body: Rest mattered more than finishing the list.
:::
```

13.39 Stims and regulation supports

Profile: `stimming-and-self-regulation@1` | Record type: `observation`

Keep a personal menu of helpful sensations and tools, plus optional notes about when they helped. The purpose is access and self-knowledge.

Writing invitation: What felt regulating, and would I like it available again?

Complete structured record: `support`. Optional fields: `context`, `effect`, `preference`, `body`.

support is repeatable. The profile has no reduction goal, compliance metric or required frequency count. The writer may prefer a static support menu with no daily monitoring at all.

```
:::record observation #example-stimming-and-self-regulation
method: stimming-and-self-regulation@1
support: Smooth stone in a pocket
context: Waiting in a noisy room
effect: Easier to stay present
:::
```

13.40 Stoic-inspired morning and evening review

Profile: `stoic-journaling@1` | Record type: `reflection`

Distinguish an event from a judgment and choose a response within personal influence. Morning intentions and evening reflection can remain simple prose.

Writing invitation: What can I influence, and what response fits my values?

Complete structured record: `event`, `response`. Optional fields: `judgment`, `within-influence`, `outside-influence`, `phase`, `learning`.

Acknowledging limited control does not excuse harm or demand emotional suppression. phase is morning or evening when supplied. Quotes should carry attribution and stay distinct from the writer's own reflections.

```
:::record reflection #example-stoic-journaling
method: stoic-journaling@1
event: The bus was late.
response: Leave a written note about the missed connection.
within-influence: How I report it
:::
```

13.41 Study, Cornell notes and retrieval practice

Profile: `study-and-note-taking@1` | Record type: `reflection`

Keep notes alongside questions that can later cue recall, plus a short summary in the writer's own words. Practice results can be logged separately.

Writing invitation: What question would test whether I understand this?

Complete structured record: `topic`, `note`, `cue`, `summary`. Optional fields: `source`, `review-on`, `recall-result`.

note and cue are repeatable. Pair cues and notes explicitly by separate records when order alone is insufficient. Review dates are suggestions, not a hidden executable spaced-repetition algorithm.

```
:::record reflection #example-study-and-note-taking
method: study-and-note-taking@1
topic: Pull-up resistors
note: They establish a default logic level.
cue: What happens when the input is otherwise floating?
summary: The resistor gives an unconnected input a defined default.
:::
```

13.42 Symptoms, sleep and care observations

Profile: `symptom-tracking@1` | Record type: `measurement`

Record an observation, unit/scale and time so it can be remembered accurately. Medication events, symptom ratings and possible contextual factors remain distinct data.

Writing invitation: What did I observe, when, and on which scale?

Complete structured record: `metric`, `status`. Optional fields: `value`, `unit`, `scale`, `at`, `recall`, `body`.

value is required for observed/estimated status and omitted for missing states. A perceived association is not causation. Standard clinical instruments require exact wording, version and permissions before an application claims compatibility.

```
:::record measurement #example-symptom-tracking
method: symptom-tracking@1
metric: sleep-duration
status: estimated
value: 7.5
unit: h
body: Approximate time asleep, not time in bed.
:::
```

13.43 Themes and directional review

Profile: `theme-system-journal@1` | Record type: `reflection`

Choose a direction for a period and notice actions that moved toward it. A theme can evolve and need not become a numerical success test.

Writing invitation: Where did I make a little more room for my theme?

Complete structured record: `theme`, `observation`. Optional fields: `period-start`, `period-end`, `action`, `alignment`, `scale`, `anti-theme`.

action is repeatable. Numeric alignment is optional and requires a scale. Partial participation remains meaningful; a theme is not automatically a pass/fail goal. Anti-theme is an optional personal label.

```
:::record reflection #example-theme-system-journal
method: theme-system-journal@1
theme: More room to breathe
observation: Left a gap between errands.
action: Paused before adding another task
:::
```

13.44 Time blocking and replanning

Profile: `time-block-planner@1` | Record type: `event`

Assign a block to an activity, revise the plan when circumstances change, and retain the distinction between intended and actual time.

Writing invitation: What is this block for, and does the remaining plan still fit?

Complete structured record: `title`, `start`, `end`, `mode`. Optional fields: `relation`, `revision`, `reason`.

mode is planned or actual. Replanned records use revises links rather than pretending the original plan happened. Overlaps are allowed with a visible warning, since real activities can overlap.

```
:::record event #example-time-block-planner
method: time-block-planner@1
title: Project reading
start: 2026-10-25T14:00:00-07:00
end: 2026-10-25T14:30:00-07:00
mode: planned
:::
```

13.45 Actual-time logs and day reconstruction

Profile: `time-logging-lifelog@1` | Record type: `session`

Describe actual activity intervals, with optional category and feeling. Retrospective reconstruction is labeled rather than presented as an exact live log.

Writing invitation: What was I actually doing, and how certain am I about the time?

Complete structured record: `activity`, `start`, `end`. Optional fields: `category`, `recall`, `confidence`, `feeling`.

The example is a completed interval; ongoing capture may omit end and use status ongoing. Weekly totals must state how overlaps, gaps and multi-category sessions are handled. Unlogged time is unknown, not wasted.

```
:::record session #example-time-logging-lifelog
method: time-logging-lifelog@1
activity: Reading on the bus
start: 2026-10-25T15:10:00-07:00
end: 2026-10-25T15:35:00-07:00
recall: retrospective
:::
```

13.46 Modular notebooks and travel-style inserts

Profile: `travelers-notebook@1` | Record type: `collection`

Group writing into replaceable inserts, topics or journeys, keeping an index and optional physical location. One .jot can hold several inserts conceptually.

Writing invitation: Which insert or journey does this belong to?

Complete structured record: `title`, `kind`. Optional fields: `member`, `physical-location`, `period-start`, `period-end`.

member is repeatable. Archiving an insert preserves cross-links and physical page locators. A digital collection does not claim exact physical compatibility with a commercial cover or refill.

```
:::record collection #example-travelers-notebook
method: travelers-notebook@1
title: Autumn carry notebook
kind: insert
physical-location: Small black notebook, second insert
:::
```

13.47 Free diary and stream of consciousness

Profile: `free-diary@1` | Record type: `reflection`

Keep narrative, fragments, contradictions and ordinary details without an imposed method.

Writing invitation: What do I want to write?

Complete structured record: `body`. Optional fields: `date`, `title`.

No date, length, mood score or conclusion is required for Core prose. This profile exists only for applications that want an explicit free-writing label.

```
:::record reflection #example-free-diary
method: free-diary@1
body: I have no plan for this paragraph. That is fine.
:::
```

13.48 Shared and dialogue journals

Profile: `shared-journal@1` | Record type: `response`

Keep each person's words and replies attributable, with a deliberate share scope.

Writing invitation: What do I want to share, and what response would I welcome?

Complete structured record: `author`, `body`. Optional fields: `relation`, `audience`, `consent`.

A response can link to an earlier record using responds-to. Sharing this file reveals its bytes; private writing requires separate access-controlled storage. No automatic invitation or message is implied.

```
:::record response #example-shared-journal
method: shared-journal@1
author: writer-b
body: I liked reading your description of the rain.
:::
```

13.49 Travel, nature and field observations

Profile: `travel-field-log@1` | Record type: `observation`

Capture what was seen, where and under what conditions, separating observations from identification guesses.

Writing invitation: What did I actually notice here?

Complete structured record: `observation`. Optional fields: `place`, `at`, `weather`, `hypothesis`, `asset`.

Precise coordinates are optional and excluded from sharing by default. Asset paths need real attachments before packaging. Uncertain identifications remain uncertain.

```
:::record observation #example-travel-field-log
method: travel-field-log@1
observation: Three small birds on the reed stems.
place: Riverside path
hypothesis: Species uncertain
:::
```

13.50 Reading and media journals

Profile: `reading-media-journal@1` | Record type: `reflection`

Connect a book, film, game or other work to a response, keeping spoilers and quotations attributable.

Writing invitation: What stayed with me from this work?

Complete structured record: `work`, `body`. Optional fields: `creator`, `locator`, `quotation`, `spoiler`, `source`.

spoiler is true or false. Repeated source and locator fields are allowed. A reaction is distinct from a factual summary, and reading progress is optional.

```
:::record reflection #example-reading-media-journal
method: reading-media-journal@1
work: A novel I am reading
body: The quiet scene mattered more than the ending.
spoiler: false
:::
```

13.51 Decision journals

Profile: `decision-journal@1` | Record type: `reflection`

Preserve the information available at decision time and compare it with a later outcome without rewriting the forecast.

Writing invitation: What do I expect, why, and when would I reconsider?

Complete structured record: `decision`, `reason`. Optional fields: `option`, `prediction`, `probability`, `review-on`, `outcome`.

option is repeatable. probability, when supplied, is numeric 0 through 1 and belongs to a specific prediction. Review records link back to the original; an outcome must not replace the original expectation.

```
:::record reflection #example-decision-journal
method: decision-journal@1
decision: Test the smaller enclosure first
reason: It will reveal the fit problem sooner.
prediction: The connector may need more clearance.
:::
```

13.52 Project and lab notebooks

Profile: `project-lab-log@1` | Record type: `observation`

Record a question, procedure, actual observation and next experiment, preserving revisions and source artifacts.

Writing invitation: What did I try, what happened, and what should I test next?

Complete structured record: `question`, `observation`. Optional fields: `procedure`, `configuration`, `asset`, `next-step`.

procedure and asset are repeatable. Configuration and units must be explicit when comparisons depend on them. A personal project log does not automatically satisfy regulated lab-record requirements.

```
:::record observation #example-project-lab-log
method: project-lab-log@1
question: Does the new cable remove the flicker?
observation: Flicker remained at the same brightness.
next-step: Inspect the supply connection.
:::
```

13.53 Food, cooking and meal memories

Profile: `food-cooking-journal@1` | Record type: `reflection`

Keep meals, recipe changes and sensory preferences as practical or autobiographical writing.

Writing invitation: What tasted good, and what would I change next time?

Complete structured record: `body`. Optional fields: `meal`, `recipe`, `ingredient`, `substitution`, `context`.

ingredient and substitution are repeatable. Nutrient estimates, if added, require separate source-labeled measurements. This profile does not require calorie tracking or label food as morally good/bad.

```
:::record reflection #example-food-cooking-journal
method: food-cooking-journal@1
```

```
body: The lentil soup was better the next day.
substitution: Used extra olives.
:::
```

13.54 Spiritual, ritual and meaning journals

Profile: `spiritual-reflection@1` | Record type: `reflection`

Allow prayer, secular ritual, values, questions and meaning-making in the writer's own language.

Writing invitation: What felt meaningful or worth sitting with?

Complete structured record: `body`. Optional fields: `tradition`, `ritual`, `question`, `source`.

No belief system, supernatural claim or preferred interpretation is imposed. Quoted sacred or literary text carries attribution; the writer may leave questions unresolved.

```
:::record reflection #example-spiritual-reflection
method: spiritual-reflection@1
body: Lighting the candle gave the evening a boundary.
ritual: A quiet minute before writing
:::
```

13.55 Memory, grief and life-story writing

Profile: `memory-life-story@1` | Record type: `reflection`

Keep a memory, the feelings around it and uncertainty about details. Several accounts may co-exist.

Writing invitation: What do I remember, and what am I unsure about?

Complete structured record: `body`. Optional fields: `person`, `period`, `confidence`, `asset`.

Remembering is not verification of an event. No generated detail should be inserted as remembered fact. Resurfacing and sharing memories are explicit choices.

```
:::record reflection #example-memory-life-story
method: memory-life-story@1
body: I remember the music more clearly than the room.
confidence: Details may have changed in memory.
:::
```

14. Implementation sequence and source register

14.1 A small, useful first implementation

Implement a writing-first reader and editor before structured forms: paragraphs, headings, date sections, scene labels, rapid logs, annotations, comments and lossless save. It must open both a one-day file and a month-long file, preserve unknown syntax and permit writing without a template. A line-by-line parser is a suitable implementation approach; styling and pagination can remain downstream, as in the separation described in the linked Fountain implementation README.

Next add optional record parsing, stable references, profile validation and portable attachment handling. Then add readable screen/PDF projections with privacy selection. Profile forms and queries come after the underlying text works reliably. Scan recognition is an optional import capability, not a prerequisite for writing a valid journal.

An implementation should expose explicit operations: open text, edit text, validate, preview, export selection, package assets and reconcile a conflict. Parsing must not send notifications, publish entries, alter the filesystem outside the chosen workspace, or run journal content. A

JavaScript parser prototype and a browser workspace now accompany this specification; full profile validation and runtime conformance are not claimed. The included fixtures and schema are implementation aids; no runtime conformance claim is made.

14.2 Writing-first quick reference

Need	Write
Ordinary journaling	Ordinary paragraphs, fragments or lists
A new day inside the current file	`### 2026-10-25 | Sunday`
A topic or method section	`####` `Gratitude` or any heading you choose
A moment/place cue	`.0830 - KITCHEN`
A small task	`[ ] Photograph the label`
A completed task	`[x] Photograph the label`
An event / note / question / idea	`o` / `-` / `?` / `!` followed by writing
A short summary	`= A small repair and a quiet evening.`
A sidebar note	`[[Remember to bring the spare cable]]`
An explicit inner voice	`::voice ME`
Literal syntax	Backslash before a leading marker, or a code fence
Reliable queryable properties	Optional `:::record ...` block
A month of writing	Keep writing under new date headings in the same file

14.3 Design decisions and compatibility boundaries

Jot keeps Fountain's readability goal and familiar sections, synopses, forced scenes, emphasis, marginalia and page breaks. It replaces screenplay-specific assumptions with journal date sections, rapid capture and optional records. Fountain compatibility is not claimed: dot-prefixed scenes, dialogue inference, title metadata and underline behavior must be handled by an explicit converter. The shared inspiration does not require inheriting a screenplay's layout or automatic uppercase character recognition.

The v1 metadata delimiters, optional record grammar and explicit voice cue are proposed decisions that make parsing deterministic. They can be revised before a stable v1 release. A stable release should freeze the grammar, publish an executable parser test suite, review all profile contracts, implement at least two independent readers, and document any interoperability issues discovered. User writing and existing original-outline sources should remain recoverable throughout that process.

14.4 Sources and scope of review

The source register below records every repository guide used for coverage and the exact commit snapshot. Review covered each guide's first 150 lines (overview, summary and available component descriptions), not an independent audit of its full bibliography. The new profiles describe representations of methods and styles; they are not copied manuals. Supplementary profiles are original Jot adaptations. Clinical profile fields are data-design choices, not clinical recommendations. Future expansion can add profiles without replacing the core writing language.

Source register

Method coverage snapshot: `shelbeeely/Journaling`, commit snapshot `f67a54dd20c9` (full hash in source links), retrieved 25 October 2026. Reviewed the first 150 lines of every method guide (overview, summary and available component descriptions), rather than independently auditing

their full research bibliographies. The catalogue is an original format adaptation based on those overviews. No repository files were changed.

Primary inspiration: Fountain repository README, read 25 October 2026. Its writing-first, readable plain-text approach informs Jot. The repository is a Fountain implementation; Jot is a separate proposed journaling format, not a Fountain-compatible screenplay dialect. No source code was copied. The original attached Jot Complete Specification v1.0 PDF is the baseline design.

Additional profiles free-diary, shared-journal, travel-field-log, reading-media-journal, decision-journal, project-lab-log, food-cooking-journal, spiritual-reflection, and memory-life-story are original complementary Jot profiles. Their presence does not imply a corresponding guide in the linked directory. The data model specifies recording behavior and makes no independent therapeutic effectiveness claims.

Repository guide mapping

1. adhd-journaling.md
2. alastair-method.md
3. art-and-junk-journaling.md
4. autistic-sensory-journaling.md
5. bullet-journal.md
6. cbt-thought-record.md
7. commonplace-book.md
8. dbt-diary-card.md
9. digital-daily-notes.md
10. dream-journal.md
11. dutch-door-method.md
12. e-ink-templates.md
13. eisenhower-matrix.md
14. emergent-task-planner.md
15. expressive-writing.md
16. five-minute-journal.md
17. five-year-diary.md
18. full-focus-planner.md
19. gender-euphoria-and-hrt-log.md
20. getting-things-done-paper.md
21. gratitude-journaling.md
22. habit-tracking.md
23. hobonichi-techo.md
24. interstitial-journaling.md
25. ivy-lee-method.md
26. jibun-techo.md
27. mood-tracking-year-in-pixels.md
28. morning-pages.md
29. paper-and-grids.md
30. passion-planner.md
31. planner-sticker-and-deco-culture.md
32. pomodoro-log.md
33. progoff-intensive-journal.md
34. prompt-journaling.md
35. rocketbook-and-scan-systems.md
36. safety-planning.md
37. shadow-growth-journal.md
38. spoon-theory-energy-accounting.md
39. stimming-and-self-regulation.md
40. stoic-journaling.md

41. study-and-note-taking.md
42. symptom-tracking.md
43. theme-system-journal.md
44. time-block-planner.md
45. time-logging-lifelog.md
46. travelers-notebook.md

15. JavaScript, diagrams, handwriting and the project roadmap

15.1 Separate the language, workspace and scanner

Jot is the journal language and file. The JavaScript parser consumes writing; a web editor provides capture and presentation; a scanner processes page images. These components can interoperate without requiring the scanner to recognize handwriting or forcing all users into one app. The accompanying parser prototype runs as an ES module in Node and browsers. It uses vendored Marked 17.0.5 for GFM Markdown and adds source-preserving journal blocks. Prototype version 0.1 is a documented subset, not a complete implementation of this draft's semantic validators. Its README and tests define its current behavior.

15.2 Diagram and chart blocks

A fenced block labeled `mermaid`, `dot`/`graphviz`, `plantuml`/`puml`, `d2`, `vega`, or `vega-lite`/`vegalite` carries diagram source. A parser identifies the engine and retains the original code block, source language and payload. Vega/Vega-Lite configuration must be a JSON object for structured interpretation; invalid input is preserved with a diagnostic. Other diagram engines can register language labels. Unknown engines remain ordinary code, making journals readable without plugins.

Recognition, syntax validation and rendering are distinct capabilities. Render adapters MUST declare the engine/version, limit resource usage, prevent unapproved remote loads and sanitize generated SVG/HTML. A remote PlantUML or other rendering service must not receive private journal text without an explicit user-selected action. Rendered output SHOULD carry a text alternative and link back to its source. The working prototype recognizes these blocks; their rendering engines are not installed yet.

15.3 Handwriting as original journal content

Handwriting does not require transcription to belong to a journal. Preserve editable strokes, a page-image fallback, page geometry and the author's intended ordering alongside typed writing. A `jot-ink` version 1 JSON asset has `format`, `version`, `page: {widthMm,heightMm}`, `coordinates: normalized`, and an ordered `strokes` list. Each stroke has `tool` (pen or eraser), `color` (#RRGGBB), `widthMm` and `points`, with point x/y/pressure values from 0 through 1. Eraser operations act on ink, not the page grid. Unknown or malformed stroke files must not execute code; import validates ranges and resource limits.

Link the stroke asset and PNG fallback through attachment records inside the same `.jot` container. A later transcription is a separate derived record with review status; it MUST NOT replace the strokes or photograph. Pressure is optional input, not a biometric identity feature. Editors SHOULD provide undo/redo, adjustable pen width, a finger/stylus mode, and non-handwriting alternatives. Stylus-only input is a basic palm-rejection aid, not a guarantee across hardware. The current canvas supports capture, stroke export/import and inserting both a PNG fallback and editable stroke data into the journal. The editor packages them together inside `.jot`; full visual block editing and handwriting recognition remain on the roadmap.

15.4 Writing assistance is optional

Spellcheck and grammar/style assistance are separate from parsing validity. An intentional fragment, dialect, name, poem or repeated word is not a malformed journal. Writing suggestions MUST be optional, non-destructive, associated with source ranges and separated from syntax diagnostics. Users should be able to ignore them without changing the journal or losing save/export capability.

The web workspace uses browser-native spellcheck. The JavaScript prototype optionally accepts a supplied lowercase word Set and produces dictionary-miss hints; it does not embed a complete spelling dictionary. The initial grammar module checks a few English patterns such as repeated words and punctuation spacing, restricted to prose rather than code or structured measurements. This is not a comprehensive grammar engine. Full grammar adapters and additional languages remain planned. External checking services require explicit opt-in before receiving journal content.

15.5 A maintained roadmap

The accompanying ROADMAP.md and the workspace's Roadmap view distinguish specified, implemented, tested, partial and planned features. Every substantial change should update the roadmap, API/version notes, acceptance criteria and known limitations together. Priorities are: reliable writing and portable export; real-device scanner/handwriting checks; profile/link validation; attachment bundles and recovery; local rendering and recognition adapters; then a stable interoperability release. No feature is reported complete merely because its profile or syntax appears in this specification.